

多波束测线路径规划问题探究

摘要

本文在多波束测深的工作原理的基础上，探究海域深度探查最优测线规划问题，要求根据不同的海域情况，建立数学模型，求解满足要求且最适合的测线方案。

问题一，建立**单线多点测深模型**，完成探测区域为海底坡面上与水平面夹角为 α 的一条线上多点测深任务。首先根据海域中心点处建立**平面直角坐标系**，由中心点处的海水深度数据以及测线间距，求解得到所有测点的海底深度数据；然后利用正弦定理求解各测点处的覆盖宽度；最后根据平坦海域重叠率的计算公式**类比推导**得到**不平坦海域重叠率**的计算公式，求解得到各测点处的重叠率。中心点处测线测得的海水深度为**70.00m**，覆盖宽度为**243.07m**，与前一条测线的重叠率为**21.26%**。

问题二，建立**多线多点测深模型**，完成矩形待测海域的测深任务。根据海底坡面、水平面、测线的空间位置关系，提取出**立体几何示意图**，参照单线多点测深模型覆盖宽度模块，并利用**三余弦定理**等边角关系推导该测深任务下各测线处覆盖宽度的表达式。根据题目提供的具体海域信息，求解得到各个位置的覆盖宽度，并探究数据规律。求解得到海域中心点处测线方向夹角为 0° 的覆盖宽度为**207.85m**。

问题三，建立**区域测深测线规划模型**，完成矩形待测区域**全覆盖测深**任务。通过几何关系把**立体模型投影到平面中**建立平面直角坐标系。求解测线及覆盖范围上下界的表达式，测线的覆盖范围在区域内为一梯形，采用**贪心思想**，相邻两条测线覆盖范围的上下界必**交于海域的边界**，以 1° 为搜索步长**遍历** β ，求解得到最优的测线路径。当 $\beta = 90^\circ$ 时，最优的测线总长度为**114724 米**，测线数量为 31 条。当 $\beta = 128^\circ$ 时，最优的测线总长度为**89246.02 米**，测线数量为 25 条。

问题四，建立**基于动态规划的多波束测线优化模型**，完成已知单波束探测数据矩形海域的**多波束探测**任务。首先通过**遍历枚举** β ，然后对海域数据作**坐标变换**，并对坐标数据进行预处理，用**插值法**对缺失数据进行补充，**动态规划**表示出最优测线规划方案。遍历 i ，比较所有满足要求的方案，求解得到最优方案：当 $\beta = 124.98^\circ$ 时，最优的测线总长度为**455134.96 米**，漏测海域占总待测海域的 2.69%，在重叠区域超过 20% 部分的总长度为 50648.66 米。

关键字：测线规划 贪心思想 动态规划 坐标变换

一、问题重述与研究背景

1.1 问题的背景

海洋占据了地球 71% 左右的面积，蕴含着丰富的资源，探测海洋地形地貌是海洋勘探中重要的一环。利用声波在水中的传播特性来测量水体的深度，常用技术有单波束探测和多波束探测。针对真实的海底地形，建立多波束探测模型，测量海底深度，对完善海洋勘探有着重要作用。

1.2 数据集说明

根据题目提供的相关信息，可知如下数据条件：

1. 一条与测线垂直平面的坡度、海水深度、多波束换能器的开角数据；
2. 一块矩形海域的坡度、中心点处的海水深度、多波束换能器的开角数据；
3. 南北长 2 海里、东西宽 4 海里矩形海域的坡度、中心点处的海水深度、多波束换能器的开角数据；
4. 南北长 5 海里、东西宽 4 海里某海域的单波束测量的测深数据。

1.3 需要解决的问题

题目要求根据不同待测海底区域，建立多波束测深模型，完成以下海底探测任务：

1. 待测海底区域为海底坡面与与测线方向垂直的平面的交线，建立模型求解多波束探测的覆盖宽度和重叠率。求解出题目给定的具体情况的结果。
2. 待测海底区域为一矩形海域，建立模型求解多波束测深覆盖宽度。求解出题目给定的具体情况的结果。
3. 待测海底区域为南北长 2 海里、东西宽 4 海里的矩形海域，根据题目给出的坡度等数据，设计出一组测量长度最短的测线对整片海域进行探测。
4. 待测海底区域为南北长 5 海里、东西宽 4 海里的矩形海域，根据该海域单波束的测深数据，设计出一组符合要求和指标的测线尽可能地完成对该片海域地探测。

二、问题分析

题目在多波束测深的工作原理的基础上，要求根据不同的海域，建立数学模型，求解满足要求的最适合的测线方案。

2.1 单线多点测深模型

建立单线多点测深模型，完成探测区域为海底坡面上与水平面夹角为 α 的一条线上多点测深任务。首先根据海域中心点处建立平面直角坐标系，由中心点处的海水深度数据以及测线间距，求解得到所有测点的海底深度数据；然后在坐标系中利用几何关系求解各测点处的覆盖宽度；最后根据平坦海域重叠率的计算公式推导得到不平坦海域重叠率的计算公式，求解得到各测点处的重叠率。

2.2 多线多点测深模型

建立多线多点测深模型，完成矩形待测海域的测深任务。根据海底坡面、水平面、测线的空间位置关系，提取出立体几何示意图，参照单线多点测深模型覆盖宽度模块，并利用三余弦定理等边角关系推导该测深任务下各测线处覆盖宽度的表达式。根据题目提供的具体海域信息，求解得到各个位置的覆盖宽度，并对数据进行分析，探究数据规律。

2.3 区域测深测线规划模型

建立区域测深测线规划模型，为矩形待测海域规划完全覆盖整片待测海域且测线总长度尽可能小的一组测线。通过几何关系求解得到测线及测线覆盖范围边界两条直线的表达式，推导得到该情况下重叠率的表达式。根据区域测深测线规划模型求解三种测线方向下各自最优的最优测线路径，比较三种情况的结果输出最优测线路径。

2.4 基于动态规划的多波束测线优化模型

建立基于动态规划的多波束测线优化模型，为已知单波束探测数据的某矩形海域设计多波束探测的最优测线路径。首先先通过遍历枚举得到最优的角度，然后对海域数据作坐标变换，将数据分为 n 列，采用动态规划表示出最优覆盖方案。遍历 i ，比较所有满足要求的方案，求解得到最优方案。

三、模型假设

1. 假设声波在海洋中不同海域的传播速度为定值。
2. 假设船的行进对声波的发射和接受没有影响。

四、符号说明

表 1 符号说明

符号	含义
D_i	第 i 个测线处的海水深度, i 为整数
α	海底坡面与水平面的夹角, 即坡度
θ	多波束换能器的开角
d	相邻两条测线的间距
W_i	第 i 个测线处的覆盖宽度
η_i	第 i 个测线与第 $i-1$ 个测线的重叠率
m_i	第 i 个测线与第 $i-1$ 个测线覆盖宽度重叠部分的长度
β	海底坡面的法向与测线方向在水平面上投影的夹角

注: 未列出符号及重复的符号以文中第一次出现处为准

五、单线多点测深模型

5.1 模型求解流程

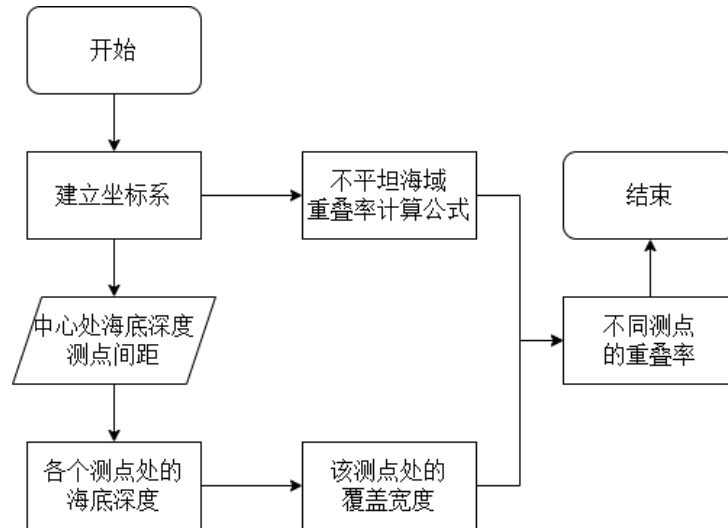


图 1 问题一求解流程图

5.2 符号定义

D_i 表示第 i 个测线处的海水深度, i 为整数; α 表示海底坡面与水平面的夹角, 即坡度; θ 表示多波束换能器的开角; d 表示相邻两条测线的间距; W_i 表示第 i 个测线处

的覆盖宽度； η_i 表示第 i 个测线与第 $i-1$ 个测线的重叠率， m_i 表示第 i 个测线与第 $i-1$ 个测线覆盖宽度重叠部分的长度。

5.3 模型建立

5.3.1 建立平面直角坐标系

首先以海域中心点与海底坡面的交点为坐标原点 O ，以水平线为 x 轴，以水平线的垂线为 y 轴，建立如下图所示坐标系。

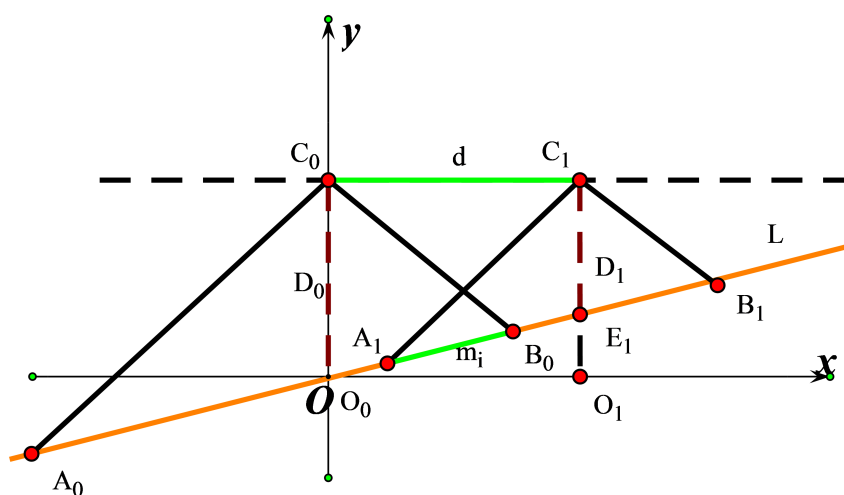


图2 平面直角坐标系示意图

已知信息： C_0 表示在中心点处的测线， C_i 表示距离中心点 id 米、即 i 个测线间距处的测线；已知坐标系中角度信息为：海底坡面与水平面的夹角为 α 、即 $\angle E_1 O_0 O_1 = \alpha$ ，多波束换能器的开角为 θ 、即 $\angle A_i B_i C_i = \theta$ ；已知坐标系中直线信息为：海底坡面和与测线方向垂直的平面的交线 $l = x \tan \alpha$ ；已知坐标系中点的坐标信息有： $O(0, 0)$ ， $C_0(0, D_0)$ ， $C_i(id, D_0)$ ， $O_i(id, 0)$ ， $C_1(d, D_0)$ ， $O_1(d, 0)$ 。

5.3.2 海水深度模块

step1. 绘制示意图：

求解第 i 个测线处的海水深度 D_i ，在上文建立直角坐标系中绘制出第 i 个测线的示意图如下图所示：

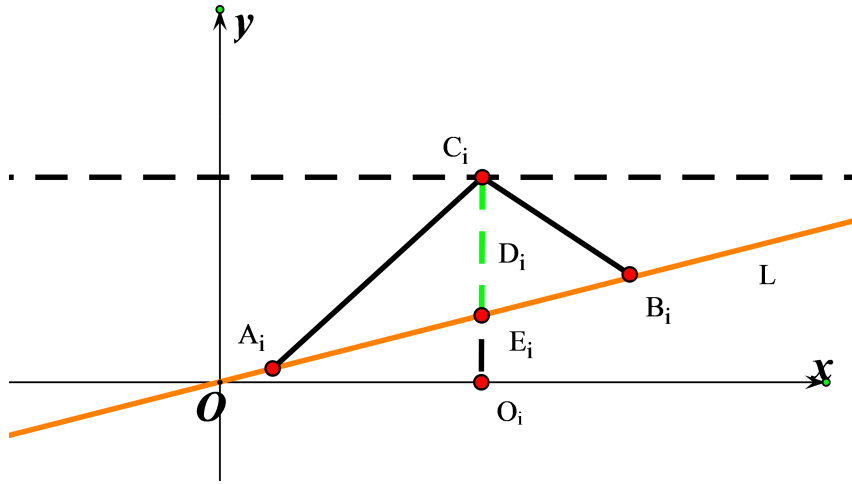


图 3 海水深度示意图

step2. 求解海水深度：

海底坡面和与测线方向垂直的平面的交线为：

$$l = x \tan \alpha \quad (1)$$

第 i 个测线在该坐标系中的坐标为 $C_i(id, D_0)$ ，可知过对应测线的点坐标作 x 轴的垂线与直线 l 交于一点，该点的坐标为 $E_i(id, id \tan \alpha)$ ，该点与测线的坐标的距离即为该点测线处的海水深度。

则各测线处海水深度为：

$$D_i = D_0 - id \tan \alpha \quad (2)$$

5.3.3 覆盖宽度模块

step1. 绘制示意图：

求解第 i 个测线处的覆盖宽度 W_i ，在上文建立直角坐标系中绘制出第 i 个测线的示意图如下图所示：

5.3.4 重叠率模块

step1. 绘制示意图：

在上文建立直角坐标系中绘制出第 i 个和第 $i-1$ 个测线的示意图如下图所示：

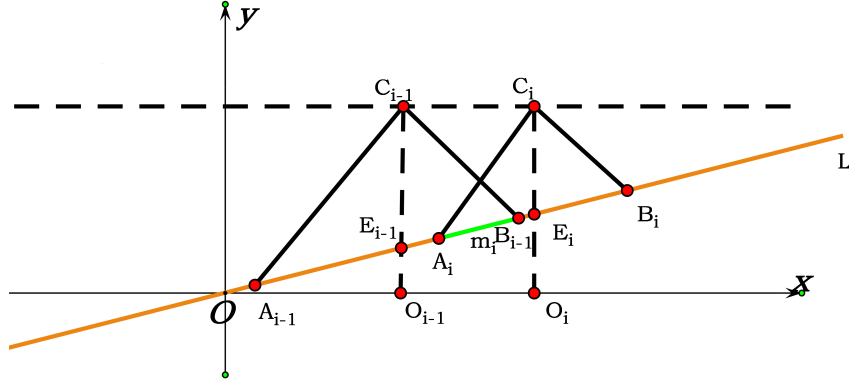


图 5 重叠率示意图

坐标 A_{i-1} 、 A_i 、 B_{i-1} 、 B_i 均在直线 $l = x \tan \alpha$ 上，可得坐标为：

$$\begin{cases} A_i(id - w_{i1}\cos\alpha, id\tan\alpha - w_{i1}\sin\alpha) \\ B_i(id + w_{i2}\cos\alpha, id\tan\alpha + w_{i2}\sin\alpha) \\ A_{i-1}((i-1)d - w_{i-11}\cos\alpha, (i-1)d\tan\alpha - w_{i-11}\sin\alpha) \\ B_{i-1}((i-1)d + w_{i-12}\cos\alpha, (i-1)d\tan\alpha + w_{i-12}\sin\alpha) \end{cases} \quad (8)$$

step.2 求解两个测线的重叠部分：

m_i 为第 i 个测线与第 $i-1$ 个测线覆盖宽度重叠部分的长度，根据两点间距离公式可以求解得到第 i 个和第 $i-1$ 个测线间重叠部分的长度为：

$$m_i = |A_i B_{i-1}| = \sqrt{(x_{A_i} - x_{B_{i-1}})^2 + (y_{A_i} - y_{B_{i-1}})^2} \quad (9)$$

step.3 推导不平坦海域重叠率计算公式：

测线相互平行且海底地形平坦海域示意图为：

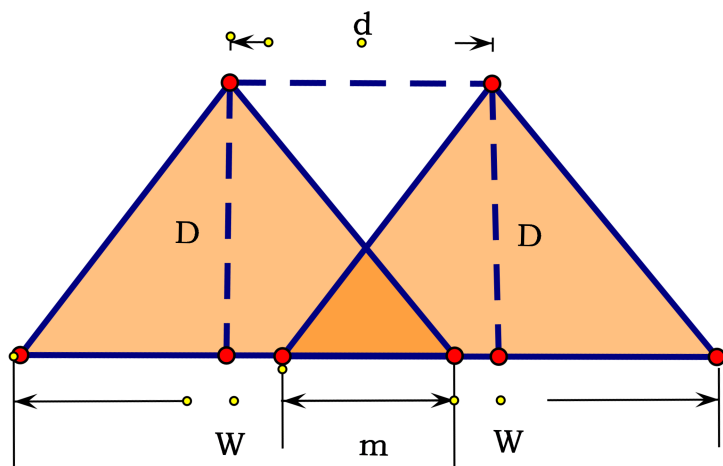


图 6 海底地形平坦海域示意图

已知测线相互平行且海底地形平坦海域的相邻条带重叠率的计算公式为：

$$\eta = 1 - \frac{d}{W} = \frac{W - d}{W} \quad (10)$$

m 表示两个测线间条带重叠部分的长度， d 表示两个测线间的距离， W 表示测线的覆盖区域。

由图 4 中的几何关系可以得到两个测线间条带重叠部分的长度为测线的覆盖区域减去两个测线间的距离，即：

$$W - d = m \quad (11)$$

则相邻条带重叠率可以表示为两个测线间条带重叠部分的长度与测线条带的覆盖区域的长度的比值，即：

$$\eta = \frac{m}{W} \quad (12)$$

可以类比得到测线相互平行但海底地形不平坦海域相邻条带重叠率的计算公式。

第 i 个测线条带与第 $i - 1$ 个测线条带之间的重叠率为：

$$\eta_i = \frac{m_i}{W_i} \quad (13)$$

根据公式 (7)、(8)、(9)、(13) 可以求解得到第 i 个测线条带与第 $i - 1$ 个测线条带之间的重叠率。

5.4 模型求解

5.4.1 待测海域信息

已知待测海域海底坡面的坡度 $\alpha = 1.5^\circ$ 、中心点处的海水深度 $D_0 = 70m$ 、多波束换能器的开角 $\theta = 120^\circ$ ，相邻两测线间距 $d = 200m$ 。

要求解海域中心点处测线以及中心点处测线左右各四个测线处的海水深度 D_i ，覆盖宽度 W_i ，测线与前一条测线的重叠率 η 。

5.4.2 求解海水深度

根据模型海水深度模块得各测线处海水深度为： $D_i = D_0 - id\tan\alpha$ 。

代入数据求解得到各测线处海水深度为：

表 2 海水深度结果

测线距中心点处的距离	-800	-600	-400	-200	0	200	400	600	800
海水深度/m	90.95	85.71	80.47	75.24	70	64.76	59.53	54.29	49.05

5.4.3 求解覆盖宽度

根据模型覆盖宽度模块得到覆盖宽度的求解公式为：

$$W_i = D_i \sin \frac{\theta}{2} \left(\frac{1}{\sin(90^\circ - \alpha - \frac{1}{2}\theta)} + \frac{1}{\sin(90^\circ + \alpha - \frac{1}{2}\theta)} \right) \quad (14)$$

代入数据求解得到各测线处覆盖宽度为：

表 3 覆盖宽度结果

测线距中心点处的距离	-800	-600	-400	-200	0	200	400	600	800
覆盖宽度/m	315.81	297.63	279.44	261.26	243.07	224.88	206.7	188.51	170.33

5.4.4 求解重叠率

根据模型重叠率模块得到：

第 i 个和第 $i-1$ 个测线间重叠部分的长度为：

$$m_i = |A_i B_{i-1}| = \sqrt{(x_{A_i} - x_{B_{i-1}})^2 + (y_{A_i} - y_{B_{i-1}})^2} \quad (15)$$

第 i 个测线条带与第 $i-1$ 个测线条带之间的重叠率为：

$$\eta_i = \frac{m_i}{W_i} \quad (16)$$

代入数据求解得到各测线处覆盖宽度为：

表 4 重叠率结果

测线距中心点处的距离	-800	-600	-400	-200	0	200	400	600	800
与前一条测线的重叠率/%	——	35.7	31.51	26.74	21.26	14.89	7.41	-1.53	-12.36

5.4.5 结果分析

根据模型计算得到问题 1 的计算结果为：

表 5 问题 1 计算结果

测线距中心点处的距离	-800	-600	-400	-200	0	200	400	600	800
海水深度/m	90.95	85.71	80.47	75.24	70	64.76	59.53	54.29	49.05
覆盖宽度/m	315.81	297.63	279.44	261.26	243.07	224.88	206.7	188.51	170.33
与前一条测线的重叠率/%	——	35.7	31.51	26.74	21.26	14.89	7.41	-1.53	-12.36

分析数据发现从左到右 9 条测线处的海水深度越来越小，预置对应的覆盖宽度也越来越小，符合事实规律。

距离中心点 600m 和 800m 处的测线与各自前一条测线的重叠率 $\eta < 0$ ，表示在这两条测线附近存在漏测的情况。

中心点左侧的测线，重叠率大都大于 20%，中心点左侧区域出现重叠过多的情况，冗余数据量大，测量效率偏低。

可以针对上面的情况进行一些测量改进，比如海水较深处测线间距稍微调大些，海水较浅处测线间距稍微调小些。这样可以在增加测量效率的同时，尽可能全面的对海域进行深度测量。

六、 多点多线测深模型

6.1 模型求解流程

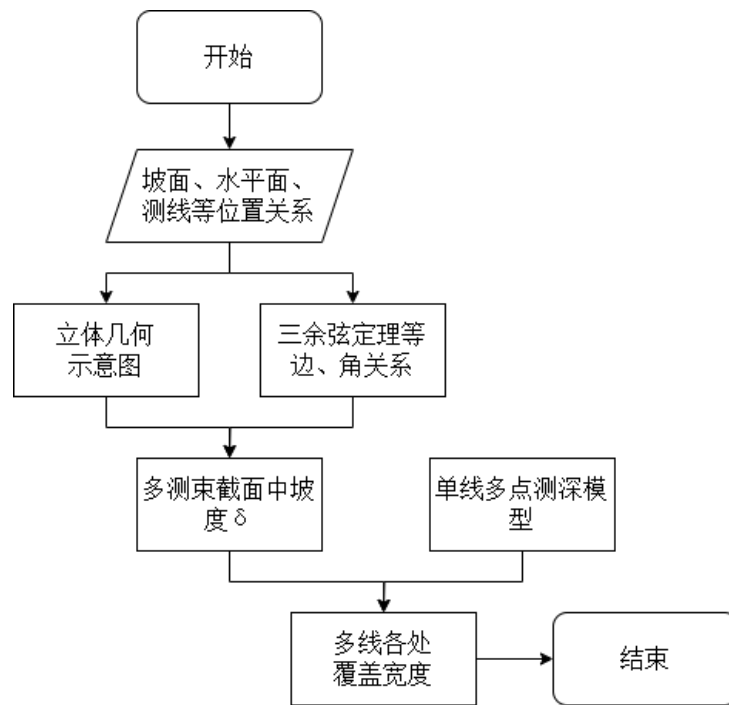


图 7 问题 2 求解流程

6.2 模型建立

6.2.1 绘制几何示意图、寻找几何关系

考虑矩形待测海域，测线经过海域中心点。根据实际场景中海底坡面、水平面、测线方向示意图，提取其中的几何关系，绘制出以下示意图：

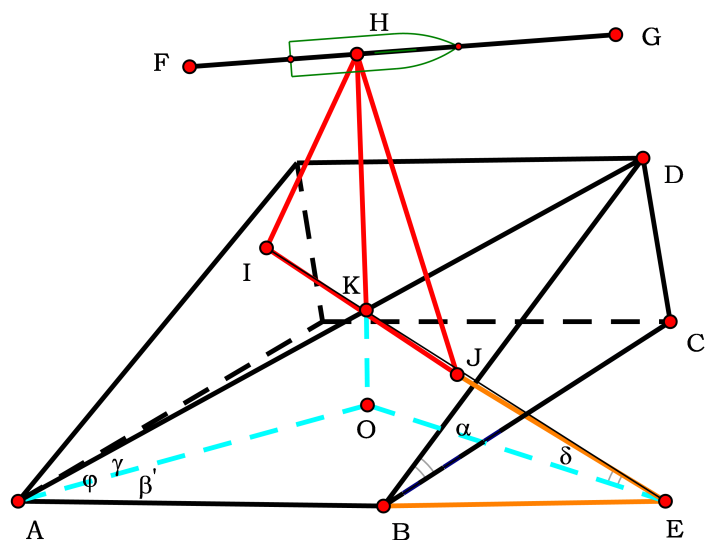


图 8 问题 2 示意图

可以得到, 平面 ABC 表示水平面, 平面 ABD 表示海底坡面, 平面 HIJ 表示航线上点 H 的多波束换能器覆盖的面。直线 FG 表示经过海域中心点的一条测线, 直线 AO 为直线 FG 在水平面上的投影。

由几何关系知, 直线 FG 垂直平面 HIJ, 直线 AB 垂直平面 BCD, 直线 AO 平行直线 FG, 所以直线 AO 垂直平面 HIJ。因为直线 OE 属于平面 HIJ, 所以直线 AO 垂直直线 OE。

海底坡面的法向与测线方向在水平面上的投影的夹角为 β , 由几何关系得 $\angle OAB = \beta' = \beta - 90^\circ$ 。海底坡面的坡度为 $\angle DBC = \alpha$ 。

设 $\angle KAO = \gamma$, $\angle KAB = \phi$, $\angle KEO = \delta$

6.2.2 三余弦定理求边角关系

将图 6 中三棱锥 D-ABC 单独拿出来, 如下图所示, 利用三余弦定理求解其中的边角关系。

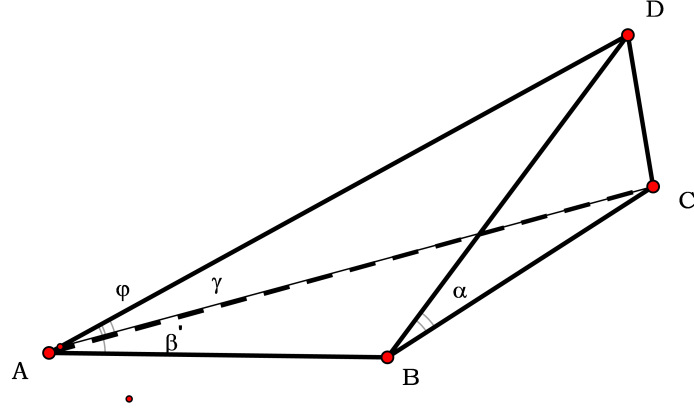


图 9 三棱锥 D-ABC

$\angle DAO = \gamma$, $\angle DAB = \phi$, $\angle OAB = \beta' = \beta - 90^\circ$, $\angle DBC = \alpha$.

设边 AB 的长度为 n , 则由于直线 AB 垂直平面 BCD, 所以边 BC 的长度为 $n \tan \beta'$;

由于直线 DC 垂直平面 ABC, 所以边 DB 的长度为 $\frac{n \tan \beta'}{\cos \alpha}$;

由于直线 AB 垂直平面 BCD, 所以边 AD 的长度为 $n \sqrt{1 + (\frac{\tan \beta'}{\cos \alpha})^2}$.

所以可得:

$$\cos \phi = \frac{1}{\sqrt{1 + (\frac{\tan \beta'}{\cos \alpha})^2}} \quad (17)$$

由三余弦定理得:

$$\cos \phi = \cos \beta' \cos \gamma \quad (18)$$

联立公式 (17)、(18) 得:

$$\cos \gamma = \frac{\cos \phi}{\cos \beta'} = \frac{1}{\cos \beta' \sqrt{1 + (\frac{\tan \beta'}{\cos \alpha})^2}} \quad (19)$$

求解得到 γ 的表达式为:

$$\gamma = \arccos \frac{\cos \phi}{\cos \beta'} = \arccos \frac{1}{\cos \beta' \sqrt{1 + (\frac{\tan \beta'}{\cos \alpha})^2}} \quad (20)$$

6.2.3 基于单线多点测深模型覆盖宽度模块求解

根据单线多点测深模型求覆盖宽度要知道图 8 中 δ 的大小。

在直角三角形 AOE 和直角三角形 KOE 中可由共边关系求解得到 δ 的大小。

由图 8 得, $|AO| = \frac{|KO|}{\tan \gamma}$, 在三角形 AOE 中 $|EO| = |AO| \tan \beta'$, 在三角形 KOE 中 $|EO| = \frac{|KO|}{\tan \delta}$.

求解得到 δ 的表达式为:

然后代入单线多点测深模型覆盖宽度模块求解覆盖宽度的大小，绘制出如下示意

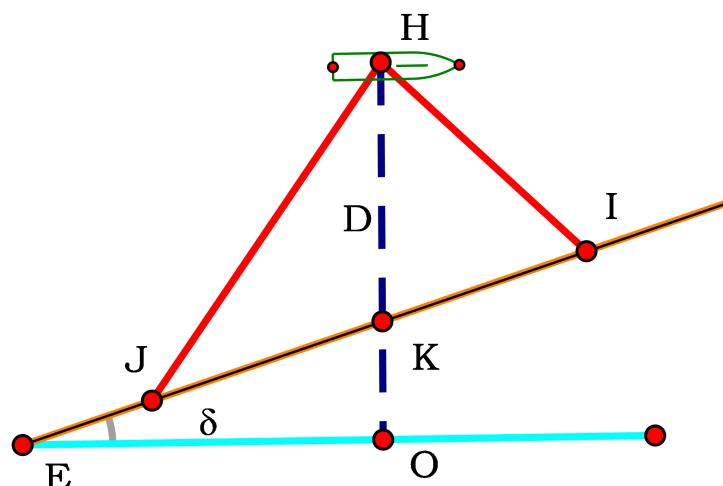


图 10 求解覆盖宽度示意图

多线多点测深模型推导得到各测线处覆盖宽度的表达式为:

联立式子 (20)、(21)、(22) 可以求解得到各测线处的覆盖宽度。

6.3 模型求解

6.3.1 待测海域信息

已知待测海域海底坡面的坡度 $\alpha = 1.5^\circ$ 、中心点处的海水深度 $D_0 = 120m$ 、多波束换能器的开角 $\theta = 120^\circ$ ，相邻两测线间距为 d 。

要求解测量船在不同角度距矩形待测海域中心点不同距离处的覆盖宽度 W 。

6.3.2 求解覆盖宽度

由表格可得 β 的值, 同时知道:

γ 的表达式为:

$$\gamma = \arccos \frac{\cos \phi}{\cos \beta'} = \arccos \frac{1}{\cos \beta' \sqrt{1 + \left(\frac{\tan \beta'}{\cos \alpha}\right)^2}} \quad (23)$$

γ 的表达式为:

$$\gamma = \arccos \frac{\cos \phi}{\cos \beta'} = \arccos \frac{1}{\cos \beta' \sqrt{1 + \left(\frac{\tan \beta'}{\cos \alpha}\right)^2}} \quad (24)$$

各测线处覆盖宽度的表达式为:

$$W_i = D_i \sin \frac{\theta}{2} \left(\frac{1}{\sin(90^\circ - \delta - \frac{1}{2}\theta)} + \frac{1}{\sin(90^\circ + \delta - \frac{1}{2}\theta)} \right) \quad (25)$$

代入数据求解得到各测线处覆盖宽度为:

表 6 问题 1 计算结果

覆盖宽度/m		测量船距离海域中心点处的距离/海里							
		0	0.3	0.6	0.9	1.2	1.5	1.8	2.1
测线 方向 夹角 /°	0	207.85	233.05	258.24	283.44	308.64	333.84	359.04	384.24
	45	416.19	451.87	487.55	523.23	558.91	594.59	630.27	665.95
	90	416.69	416.69	416.69	416.69	416.69	416.69	416.69	416.69
	135	416.19	380.51	344.83	309.15	273.47	237.79	202.11	166.43
	180	207.85	182.65	157.45	132.25	107.05	81.85	56.65	31.45
	225	416.19	380.51	344.83	309.15	273.47	237.79	202.11	166.43
	270	416.69	416.69	416.69	416.69	416.69	416.69	416.69	416.69
	315	416.19	451.87	487.55	523.23	558.91	594.59	630.27	665.95

6.3.3 结果分析

考虑待测海域以及测线的分布情况,可以发现结果表格中每一行的数据都是船只在中心点位置按照与不同的 β 角,从海域中心向外延伸的测线。表格内 8 行数据其实可以看作由 4 条相交于海域中心点的测线得到的。

根据覆盖宽度,将覆盖宽度结果可视化绘制出测线在海域底部的俯视图,如下图所示:

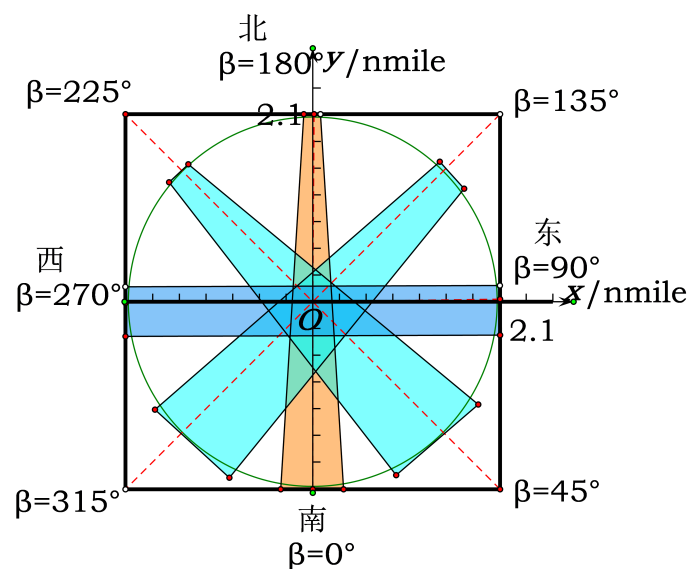


图 11 测线覆盖轨迹俯视图

分析 $\beta = 0^\circ$ 和 $\beta = 180^\circ$ 的深度数据可以发现从这片海域正北方已知往南，测线覆盖宽度单调递增，说明这片海域是北浅南深的。

分析 $\beta = 90^\circ$ 和 $\beta = 270^\circ$ 的深度数据可以发现从这片海域正西方已知往东，测线覆盖宽度一直为 416.69m，说明这片海域从东到西海水深度一样。

从这两个规律可以发现与题目所给海域地形一致，所得结果符合事实，可信性强。

七、区域测深测线规划模型

7.1 模型求解流程

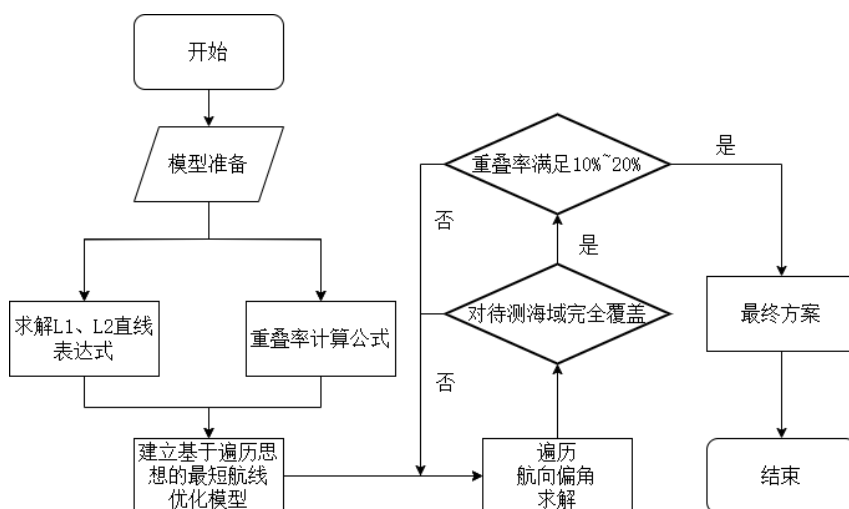


图 12 问题三求解流程图

7.2 模型准备工作

7.2.1 求解覆盖宽度 W

step.1 绘制立体示意图:

已知海域西深东浅，绘制出海域示意图如下图所示：

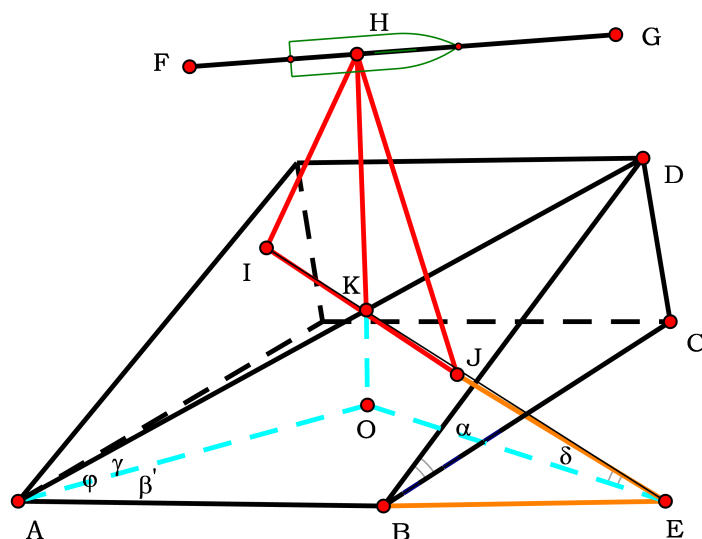


图 13 立体示意图

设 $|AO| = x_0$, 则 $|OK| = x_0 \tan \gamma$, $|OH| = D + x_0 \tan \gamma$, $|OE| = x_0 \tan \beta'$

step2. 建立坐标系 1:

在平面 HIJ 上以 E 点为坐标原点，直线 OE 为 x 轴，过点 E 垂直与直线 OE 的直线为 y 轴建立平面直角坐标系，如图所示：

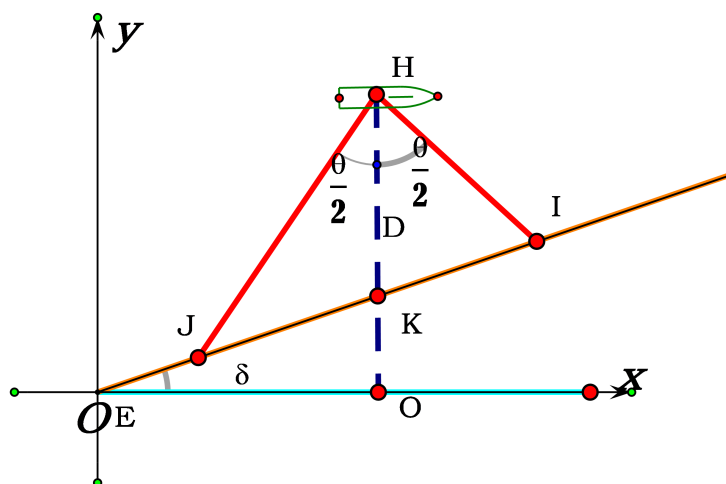


图 14 坐标系 1

可以得到坐标 $H(x_0 \tan \beta' + \Delta x, D + x_0 \tan \gamma)$ 、 $K(x_0 \tan \beta' + \Delta x, x_0 \tan \gamma)$
在坐标系中设任意一条经过 H 点的直线表达式为：

$$y = k(x - x_0) + D + x_0 \tan \gamma \quad (26)$$

于是可以得到直线 HJ、HI 的表达式为：

$$l_{HJ} : y = \tan(90^\circ - \frac{\theta}{2})(x - (x_0 \tan \beta' + \Delta x)) + D + x_0 \tan \gamma \quad (27)$$

$$l_{HI} : y = -\tan(90^\circ - \frac{\theta}{2})(x - (x_0 \tan \beta' + \Delta x)) + D + x_0 \tan \gamma \quad (28)$$

又因为直线 IJ 表达式为：

$$l_{IJ} : y = \tan \delta x \quad (29)$$

将直线 IJ 分别与直线 HJ、HI 联立可以求解得到 I、J 点的横坐标为：

$$\begin{cases} x_I = \frac{h + x_0 \tan \beta' \tan(90^\circ - \frac{\theta}{2})}{\tan \delta + \tan(90^\circ - \frac{\theta}{2})} \\ x_J = \frac{h - x_0 \tan \beta' \tan(90^\circ - \frac{\theta}{2})}{\tan \delta - \tan(90^\circ - \frac{\theta}{2})} \end{cases} \quad (30)$$

可以求解得到 w_1, w_2 的表达式为：

$$\begin{cases} w_1 = |JK| = \frac{1}{\cos \delta} |x_J - x_K| \\ w_2 = |IK| = \frac{1}{\cos \delta} |x_I - x_K| \end{cases} \quad (31)$$

可以将 w_1, w_2 表示为：

$$\begin{cases} w_1 = \mu'_1 x_0 + \lambda'_1 \\ w_2 = \mu'_2 x_0 + \lambda'_2 \end{cases} \quad (32)$$

截距 Δx 的表达式为：

$$\Delta x = \frac{b}{\tan \phi} \sin \beta' \quad (33)$$

$\mu'_1, \mu'_2, \lambda'_1, \lambda'_2$ 的表达式为：

$$\begin{cases} \mu'_1 = (\frac{\tan \beta' \tan(90^\circ - \frac{\theta}{2})}{\tan \delta - \tan(90^\circ - \frac{\theta}{2})} + \tan \beta') \frac{1}{\cos \delta} \\ \mu'_2 = (\frac{\tan \beta' \tan(90^\circ - \frac{\theta}{2})}{\tan \delta - \tan(90^\circ - \frac{\theta}{2})} - \tan \beta') \frac{1}{\cos \delta} \\ \lambda'_1 = \frac{[\Delta x - (h - \tan(90^\circ - \frac{\theta}{2}) \Delta x)]}{\tan \delta - \tan(90^\circ - \frac{\theta}{2})} \frac{1}{\cos \delta} \\ \lambda'_2 = \frac{[(h + \tan(90^\circ - \frac{\theta}{2}) \Delta x) - \Delta x]}{\tan \delta + \tan(90^\circ - \frac{\theta}{2})} \frac{1}{\cos \delta} \end{cases} \quad (34)$$

step.3 以海底坡面建立坡面直角坐标系

在海底坡面上以 A 为坐标原点，直线 AB 为 x 轴，过 A 点作直线 AB 的垂线为 y 轴，建立坡面直角坐标系，如下图所示：

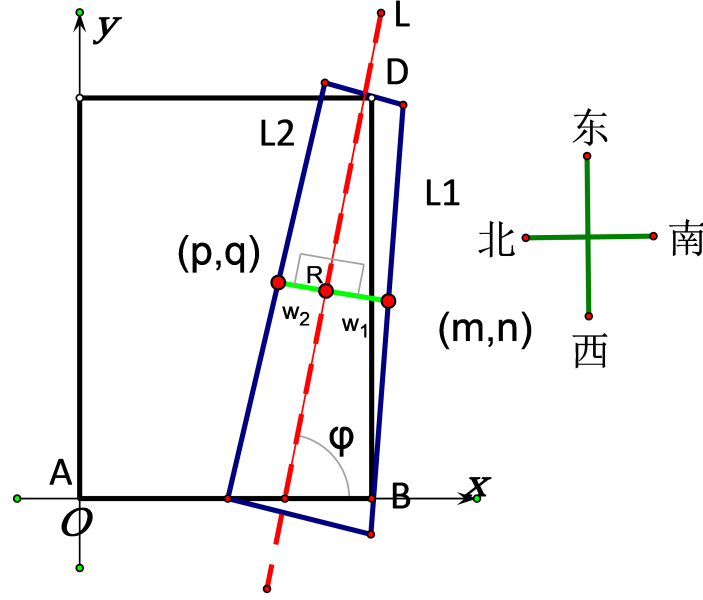


图 15 海底坡面坐标系 2

在立体解析图中，可以得到以下关系式：

$$\begin{cases} x_0 = x \cos \beta' \\ w_1 = \mu'_1 x_0 + \lambda'_1 = \mu'_1 x \cos \beta' + \lambda'_1 \\ \mu_1 = \mu'_1 \cos \beta' = \left(\frac{\tan \beta' \tan(90^\circ - \frac{\theta}{2})}{\tan \delta - \tan(90^\circ - \frac{\theta}{2})} + \tan \beta' \right) \frac{\cos \beta'}{\cos \delta} \\ \mu_2 = \mu'_2 \cos \beta' = \left(\frac{\tan \beta' \tan(90^\circ + \frac{\theta}{2})}{\tan \delta - \tan(90^\circ - \frac{\theta}{2})} - \tan \beta' \right) \frac{\cos \beta'}{\cos \delta} \\ \lambda_1 = \lambda'_1 = \frac{[\Delta x - (h - \tan(90^\circ - \frac{\theta}{2}) \Delta x)]}{\tan \delta - \tan(90^\circ - \frac{\theta}{2})} \frac{1}{\cos \delta} \\ \lambda_2 = \lambda'_2 = \frac{[(h + \tan(90^\circ - \frac{\theta}{2})) \Delta x - \Delta x]}{\tan \delta + \tan(90^\circ - \frac{\theta}{2})} \frac{1}{\cos \delta} \end{cases} \quad (35)$$

可以将 w_1, w_2 在坐标系 2 中表示为：

$$\begin{cases} w_1 = \mu_1 x + \lambda_1 \\ w_2 = \mu_2 x + \lambda_2 \end{cases} \quad (36)$$

step4. 以海底水平面建立平面直角坐标系 3

在水平面 ABC 内，以 A 为坐标原点，直线 AB 为 x 轴，在平面 ABC 内过 A 点的垂线为 y 轴建立平面直角坐标系 3 如下图所示。

化简得 w_1 的表达式为：

$$w_1 = \frac{\mu_1 \cos \delta}{1 + \sin \beta' \mu_1 \cos \delta} m + \frac{\lambda_1 \cos \delta}{1 + \sin \beta' \mu_1 \cos \delta} \quad (40)$$

将点 R 的坐标代入直线 l 的表达式中得到：

$$n + w_1 \cos \beta' = \tan \beta' (m - w_1 \sin \beta') + b \quad (41)$$

将 w_1 的表达式代入上式中得：

$$n = (\tan \beta' - \frac{\tan \beta' \sin \beta' \mu_1 \cos \delta}{1 + \sin \beta' \mu_1 \cos \delta} - \frac{\cos \beta' \mu_1 \cos \delta}{1 + \sin \beta' \mu_1 \cos \delta}) m + b'_1 \quad (42)$$

$$b'_1 = -\frac{\tan \beta' \sin \beta' \lambda_1 \cos \delta}{1 + \sin \beta' \mu_1 \cos \delta} - \frac{\lambda_1 \cos \delta \cos \beta'}{1 + \sin \beta' \mu_1 \cos \delta} + b \quad (43)$$

坐标 (m, n) 为直线 l_1 上任意的一点，所以可以得到直线 l_1 的表达式为：

$$y = (\tan \beta' - \frac{\tan \beta' \sin \beta' \mu_1 \cos \delta}{1 + \sin \beta' \mu_1 \cos \delta} - \frac{\cos \beta' \mu_1 \cos \delta}{1 + \sin \beta' \mu_1 \cos \delta}) x + b'_1 \quad (44)$$

$$b'_1 = -\frac{\tan \beta' \sin \beta' \lambda_1 \cos \delta}{1 + \sin \beta' \mu_1 \cos \delta} - \frac{\lambda_1 \cos \delta \cos \beta'}{1 + \sin \beta' \mu_1 \cos \delta} + b \quad (45)$$

同理可以得到直线 l_2 的表达式为：

$$y = (\tan \beta' + \frac{\tan \beta' \sin \beta' \mu_2 \cos \delta}{1 - \sin \beta' \mu_2 \cos \delta} + \frac{\cos \beta' \mu_2 \cos \delta}{1 - \sin \beta' \mu_2 \cos \delta}) x + b'_2 \quad (46)$$

$$b'_2 = \frac{\tan \beta' \sin \beta' \lambda_2 \cos \delta}{1 - \sin \beta' \mu_2 \cos \delta} - \frac{\lambda_2 \cos \delta \cos \beta'}{1 - \sin \beta' \mu_2 \cos \delta} + b \quad (47)$$

7.2.3 求解重叠率

求解第 i 条测线与第 $i+1$ 条测线之间的重叠率，在水平坐标系中绘制出第 i 条测线与第 $i+1$ 条测线在水平面上的投影，如下图所示：

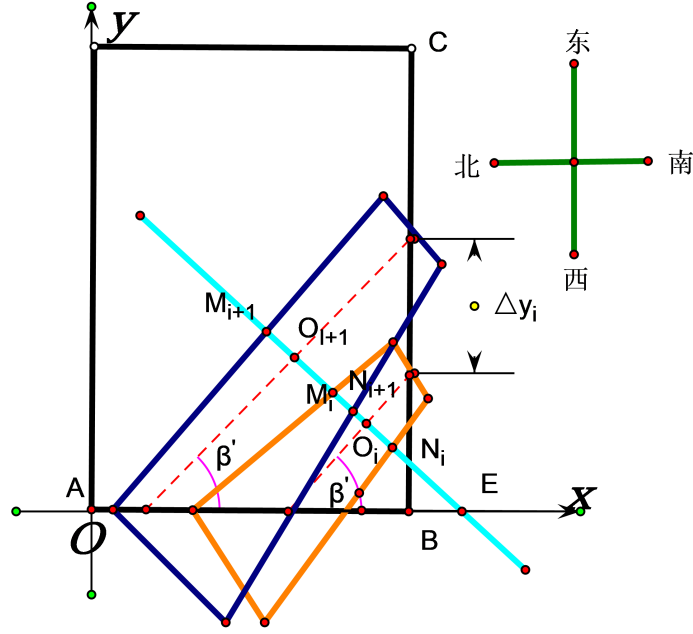


图 17 测线在水平面投影示意图

第 $i+1$ 条测线是由第 i 条测线沿 y 轴平移 Δy_i 得到的，如上图所示。

船在第 i 条测线上第 j 个位置处在水平面的投影点为 $O_{ij}(x_{ij}, y_{ij})$

直线 $O_{ij}E$ 的表达式为：

$$O_{ij}E: \quad y = -\frac{1}{\tan\beta'}(x - x_{ij}) + y_{ij} \quad (48)$$

由几何关系得以下边角关系：

$$\begin{cases} |O_{ij}O_{i+1,j+1}| = \cos\beta' \Delta y_i \\ x_{M_i} = x_{O_i} - d_2(x_{O_i}) \sin\beta' \\ x_{N_i} = x_{O_i} + d_1(x_{O_i}) \sin\beta' \\ x_{N_{i+1}} = x_{O_{i+1}} + d_1(x_{O_{i+1}}) \sin\beta' \\ x_{O_{i+1}} = x_{O_i} - \Delta y_i \sin\beta' \cos\beta' \end{cases} \quad (49)$$

所以可以得到相邻两个测线间重叠部分的长度为：

$$m_i = |M_i N_{i+1}| = \frac{x_{N_{i+1}} - x_{M_i}}{\sin\beta'} \quad (50)$$

第 $i+1$ 个测线的覆盖宽度为：

$$W_{i+1} = |M_{i+1} N_{i+1}| = d_1(x_{O_{i+1}}) + d_2(x_{O_{i+1}}) \quad (51)$$

所以第 $i+1$ 条测线覆盖宽度与第 i 条测线覆盖宽度的重叠率为：

$$\eta_{i+1} = \frac{m_i}{W_{i+1}} = \frac{|M_i N_{i+1}|}{|M_{i+1} N_{i+1}|} \quad (52)$$

7.3 建立区域测深测线规划模型

7.3.1 目标函数：

设计的一组测线要满足测线的总长度最短：

$$\min \sum_{i=1}^N L_i \quad (53)$$

7.3.2 决策变量：

海底坡面的法向与测线方向在水平面上投影的夹角为 β , $\beta' = \beta - 90^\circ$.

测线的方向满足条件： $\beta' \in [0^\circ, 90^\circ]$.

在寻找最优 β' 时根据模型准备阶段的推导将不同的测线方法分为三种情况求解：

$$\beta' = \begin{cases} 0^\circ & \text{测线方向为南北走向} \\ 90^\circ & \text{测线方向为东西走向} \\ (0^\circ, 90^\circ) & \text{其它测线方向} \end{cases} \quad (54)$$

7.3.3 约束条件：

建立如下的约束条件：

约束条件 1： 不同测线方向和不同测线与坐标轴的截距 b_i 对应的测线长度。

$$L_i = \begin{cases} \frac{2}{\cos\beta'} + \frac{b_i}{\sin\beta'} & \tan\beta' < 2, & b_i < 0 \\ \frac{2}{\cos\beta'} & \tan\beta' < 2, & 0 \leq b_i < 4 - 2\tan\beta' \\ \frac{4-b_i}{\sin\beta'} & \tan\beta' < 2, & b_i > 4 - 2\tan\beta' \\ \frac{4-b_i}{\sin\beta'} & \tan\beta' \geq 2, & b_i > 0 \\ \frac{4}{\sin\beta'} & \tan\beta' \geq 2, & 2\tan\beta' - 4 < b_i \leq 0 \\ \frac{2}{\cos\beta'} + \frac{b_i}{\sin\beta'} & \tan\beta' \geq 2, & b_i < 2\tan\beta' - 4 \end{cases} \quad (55)$$

约束条件 2： 相邻条带之间的重叠率满足 $10\% \leq \eta \leq 20\%$ 的要求：

$$\begin{cases} \min(\eta_i) \geq 10\% \\ \max(\eta_i) \leq 20\% \end{cases} \quad (56)$$

约束条件 3： 矩形海域南北长 2 海里、东西宽 4 海里。

$$\sum_{i=1}^{N-1} \Delta y_i \cos^2 \beta' \geq 4 \quad (57)$$

$$\sum_{i=1}^{N-1} \Delta y_i \cos \beta' \sin \beta' \geq 2 \quad (58)$$

7.3.4 总模型

综上所述，建立如下关于区域测深测线规划模型：

$$\begin{aligned} \min \quad & \sum_{i=1}^N L_i \\ \text{s.t.} \quad & \left\{ \begin{array}{ll} \frac{2}{\cos \beta'} + \frac{b_i}{\sin \beta'} & \tan \beta' < 2, \quad b_i < 0 \\ \frac{2}{\cos \beta'} & \tan \beta' < 2, \quad 0 \leq b_i < 4 - 2 \tan \beta' \\ \frac{4-b_i}{\sin \beta'} & \tan \beta' < 2, \quad b_i > 4 - 2 \tan \beta' \\ \frac{4-b_i}{\sin \beta'} & \tan \beta' \geq 2, \quad b_i > 0 \\ \frac{4}{\sin \beta'} & \tan \beta' \geq 2, \quad 2 \tan \beta' - 4 < b_i \leq 0 \\ \frac{2}{\cos \beta'} + \frac{b_i}{\sin \beta'} & \tan \beta' \geq 2, \quad b_i < 2 \tan \beta' - 4 \end{array} \right. \\ & \left\{ \begin{array}{l} \min(\eta_i) \geq 10\% \\ \max(\eta_i) \leq 20\% \end{array} \right. \\ & \sum_{i=1}^{N-1} \Delta y_i \cos^2 \beta' \geq 4 \\ & \sum_{i=1}^{N-1} \Delta y_i \cos \beta' \sin \beta' \geq 2 \end{aligned} \quad (60)$$

7.4 模型求解

7.4.1 求解步骤

step1 枚举 β' 的角度，以 1° 为步长在 $\beta' \in (0^\circ, 90^\circ)$ 范围内遍历；

step2 特殊处理 0° 和 90° 的情况，得到对应的最优规划路线；

step3 由区域测深测线规划模型可知，在 β' 角度确定的情况下，轨迹 l , l_1 , l_2 , w_1 , w_2 的斜率为定值，故求得相应的斜率，参照下图

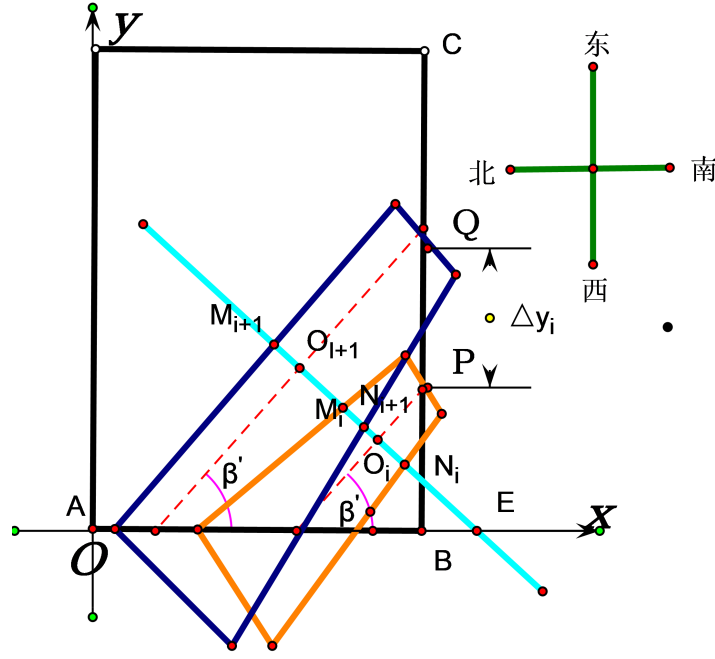


图 18 求解示意图

step4 基于贪心思想，默认覆盖率最小的情况下，我们认为当前航向的下界线 l_1 一定会经过上条航线上界线 l_2 的边界点。判断 l_1, l_2 的斜率正负性，若为正则从起始点 $(L, 0)$ 开始递推，反之则从起始点 (L, H) 开始递推。

step5 设当前的边界点为 P ，定义为上一条航线的 l_2 与海域边界的第一象限交点由模型可知， l_1, l_2, w_1, w_2 的截距由 l 的截距决定，故采用二分答案的方式来确定 l 的截距 b ，使得 l_1 过当前边界点 P ，如图 18 所示。

step6 由 step5 得到的 b 来求解得到此次航线的轨迹 l_1 ，以及 l_1, l_2 ；

step7 把 l 投影到水平面和上一条轨迹在水平面，用公式 (51) 求得重叠率，若大于 20%，则回到 step1，若小于 10% 则适当减小 b ，回到 step5；

step8 用 l_2 计算下一条航线的边界点 Q ，回到 step5，直到 l_2 与航线无交点，如图 18 所示。

step9 记录方案，并根据枚举的角度寻优。

7.4.2 $\beta' = 0^\circ$ 时规划最优测线路径

当 $\beta' = 0^\circ$ 时， $\beta = 90^\circ$ ，求解得到此种情况下最优的测线总长度为 114824 米，测线的数量为 31 条，此时的测线路径为：

表 7 $\beta = 90^\circ$ 时的最优规划路线

测线起点	(0,358.52)	(0,1013.34)	(0,1611.34)	(0,2157.45)	(0,2656.17)	(0,3111.62)	(0,3527.54)	(0,3907.37)
测线终点	(3704.00,358.52)	(3704.00,1013.34)	(3704.00,1611.34)	(3704.00,2157.45)	(3704.00,2656.17)	(3704.00,3111.62)	(3704.00,3527.54)	(3704.00,3907.37)
测线起点	(0,4254.24)	(0,4571.01)	(0,4860.29)	(0,5124.48)	(0,5365.73)	(0,5586.05)	(0,5787.26)	(0,5971.00)
测线终点	(3704.00,4254.24)	(3704.00,4571.01)	(3704.00,4860.29)	(3704.00,5124.48)	(3704.00,5365.73)	(3704.00,5586.05)	(3704.00,5787.26)	(3704.00,5971.00)
测线起点	(0,6138.80)	(0,6292.04)	(0,6431.98)	(0,6559.78)	(0,6676.49)	(0,6783.07)	(0,6880.40)	(0,6969.29)
测线终点	(3704.00,6138.80)	(3704.00,6292.04)	(3704.00,6431.98)	(3704.00,6559.78)	(3704.00,6676.49)	(3704.00,6783.07)	(3704.00,6880.40)	(3704.00,6969.29)
测线起点	(0,7050.46)	(0,7124.59)	(0,7192.29)	(0,7254.11)	(0,7310.57)	(0,7362.12)	(0,7409.21)	
测线终点	(3704.00,7050.46)	(3704.00,7124.59)	(3704.00,7192.29)	(3704.00,7254.11)	(3704.00,7310.57)	(3704.00,7362.12)	(3704.00,7409.21)	

表格内坐标表示以海底坡面建立的坐标系内的数据。

将测线路径可视化如下图所示：

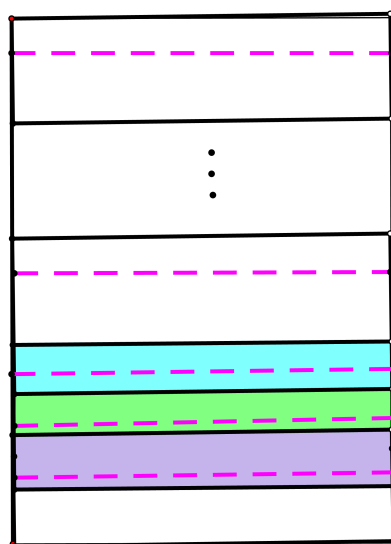


图 19 $\beta = 90^\circ$ 时的最优规划路线

7.4.3 $\beta' = 90^\circ$ 时规划最优测线路径

当 $\beta' = 90^\circ$ 时， $\beta = 180^\circ$ ，求解得到此种情况下最优的测线总长度为 608965.4 米，测线的数量为 82 条。

将测线路径可视化如下图所示：

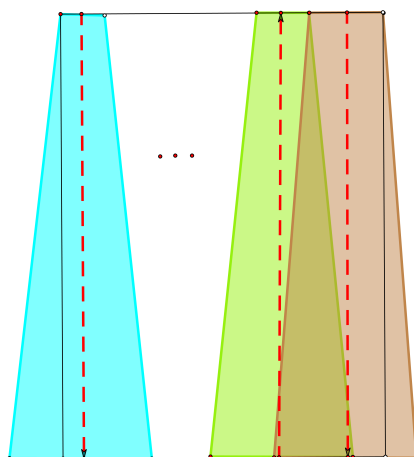


图 20 $\beta = 180^\circ$ 时的最优规划路线

7.4.4 $\beta' \in (0^\circ, 90^\circ)$ 时规划最优测线路径

当 $\beta' \in (0^\circ, 90^\circ)$ 时，从 1° 到 89° 以 1° 遍历，求解探测路径。比较结果得到，当 $\beta' = 38^\circ$ 时， $\beta = 128^\circ$ ，求解得到此种情况下最优的测线总长度为 89246.02 米，测线的数量为 25 条。此时的探测路径为：

表 8 $\beta = 128^\circ$ 时的最优规划路线

航线起点	(2744.11,0.00)	(1850.50,0.00)	(1018.59,0.00)	(244.11,0.00)	(0.00,372.71)	(0.00,897.30)	(0.00,1385.67)
航线终点	(3704.00,750.20)	(3704.00,1448.61)	(3704.00,2098.79)	(3704.00,2704.09)	(3704.00,3267.58)	(3704.00,3792.17)	(3704.00,4280.54)
航线起点	(0.00,1840.32)	(0.00,2263.58)	(0.00,2657.61)	(0.00,3024.44)	(0.00,3365.94)	(0.00,3683.86)	
航线终点	(3704.00,4735.19)	(3704.00,5158.45)	(3704.00,5552.48)	(3704.00,5919.31)	(3704.00,6260.81)	(3704.00,6578.73)	
航线起点	(0.00,3979.82)	(0.00,4255.36)	(0.00,4511.87)	(0.00,4764.96)	(0.00,5029.94)	(0.00,5307.44)	
航线终点	(3704.00,6874.70)	(3704.00,7150.23)	(3704.00,7406.74)	(3381.78,7408.00)	(3042.74,7408.00)	(2687.68,7408.00)	
航线起点	(0.00,5598.05)	(0.00,5902.40)	(0.00,6221.14)	(0.00,6554.93)	(0.00,6904.50)	(0.00,7270.59)	
航线终点	(2315.83,7408.00)	(1926.42,7408.00)	(1518.60,7408.00)	(1091.50,7408.00)	(644.23,7408.00)	(175.81,7408.00)	

表格内坐标表示以海底坡面建立的坐标系内的数据。

将测线路径可视化如下图所示：

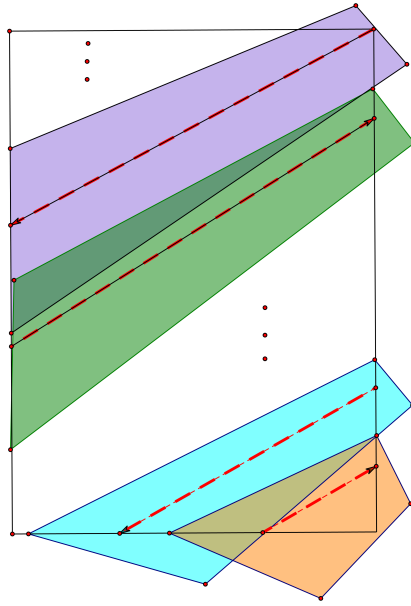


图 21 $\beta = 128^\circ$ 时的最优规划路线

7.4.5 结果分析

最终比较上述三种情况，得到当 $\beta = 128^\circ$ 时可以求得整个待测矩形海域的最优一组测线方案。测线总长度为 89246.02 米，测线的数量为 25 条。此时的探测路径由上表所示。

八、基于动态规划的多波束测线优化模型

8.1 模型建立流程

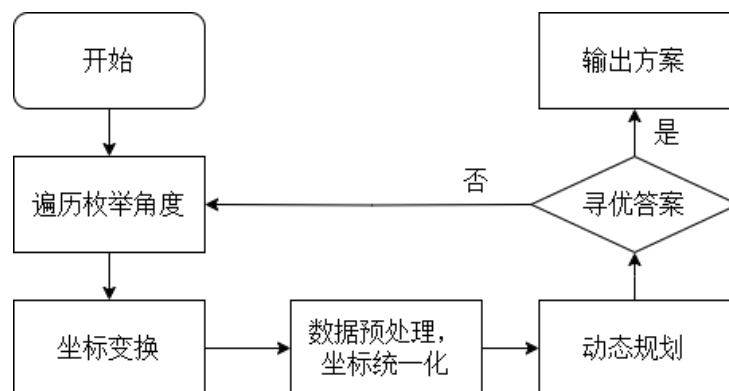


图 22 问题 4 求解流程

8.2 准备工作

已知该海域单波束测量海水深度数据，将数据可视化得到，该海底坡面的三维图如下图所示：

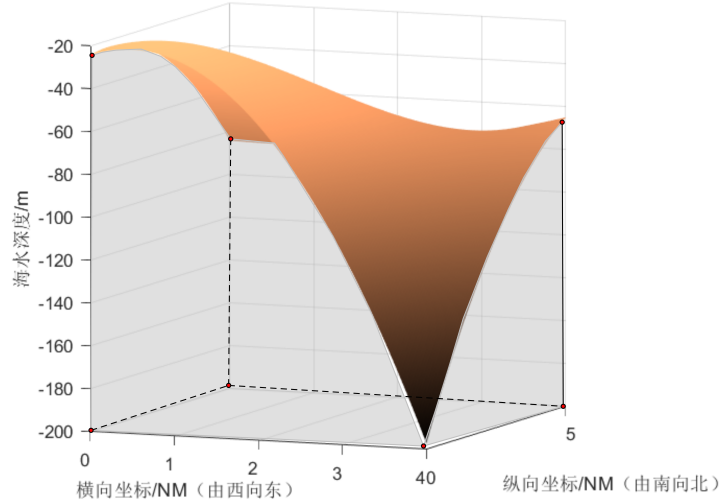


图 23 海底坡面的三维图

以该矩形海域水平面建立平面直角坐标系，海域左下角顶点为坐标原点，已知该海域内各点的横向坐标和纵向坐标。

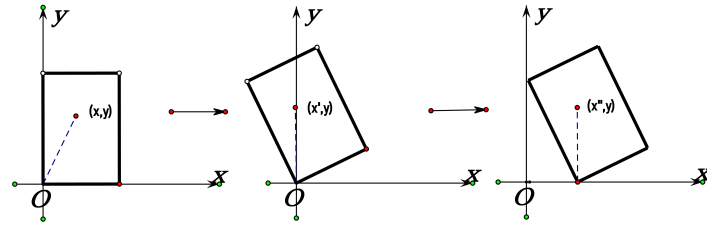


图 24 海域数据坐标转换

将该海域整体数据以坐标原点为圆心逆时针旋转 α 角，得到各个对应点的新坐标为：

$$\begin{bmatrix} \cos\alpha & \sin\alpha \\ -\sin\alpha & \cos\alpha \end{bmatrix} \begin{bmatrix} x & y \end{bmatrix} = \begin{bmatrix} x' & y' \end{bmatrix} \quad (61)$$

再将数据整体像右平移 $|x - x'|$ 个单位长度，得到各个对应点的新坐标为 (x'', y'') 。
其中 $x'' = x' + a\cos\alpha$, $y'' = y'$ 。

8.3 模型建立

8.3.1 目标函数

已知测线探测重叠的面积为：

$$S_q = h \sum_{i=1}^N \sum_{j=i+1}^N (l_i + l_j - (x_i - x_j)) \quad (62)$$

测线探测的海底的总面积为：

$$S_{sum} = h \sum_{i=1}^N l_i + h \sum_{j=i+1}^N l_j - S_q \quad (63)$$

测线未探测到的海底的总面积为：

$$S_d = M(i, j) - S_{sum} \quad (64)$$

在满足覆盖率大于 90% 的前提下，总测线的距离尽可能小，再考虑重叠率尽可能控制在 20% 以下。得到三目标函数：

$$\min \frac{S_d}{S_{sum}} \quad (65)$$

$$\min \sum_{i=1}^N L_i \quad (66)$$

$$\min \eta_i = \frac{(l_i + l_j - (x_i - x_j))}{l_i} \quad (67)$$

8.3.2 决策变量

海底坡面的法向与测线方向在水平面上投影的夹角为 β ， $\beta' = \beta - 90^\circ$ 。

测线的方向满足条件： $\beta' \in [0^\circ, 90^\circ]$ 。

在寻找最优 β' 时根据模型准备阶段的推导将不同的测线方法分为三种情况求解：

$$\beta = (0^\circ, 360^\circ) \quad \text{测线方向} \quad (68)$$

8.3.3 约束条件

约束条件 1：i 的宽度：

$$l_i = d_i^1 + d_{i+1}^2 \quad (69)$$

约束条件 2: j 的宽度:

$$l_j = d_j^1 + d_{j-1}^2 \quad (70)$$

约束条件 3: 测线未探测到的区域占据探测到的区域的 0.05

$$\frac{S_d}{S_{sum}} < 5\% \quad (71)$$

约束条件 4: 单条测线的长度为:

$$L_i = \max(y_i^1) - \min(y_i^1) \quad (72)$$

8.3.4 总模型

综上所述, 建立如下关于基于动态规划的多波束测线优化模型:

$$\min \frac{S_d}{S_{sum}} \quad (73)$$

$$\min \sum_{i=1}^N L_i \quad (74)$$

$$\min \eta_i = \frac{(l_i + l_j - (x_i - x_j))}{l_i} \quad (75)$$

$$\begin{cases} l_i = d_i^1 + d_{i+1}^2 \\ l_j = d_j^1 + d_{j-1}^2 \\ \frac{S_d}{S_{sum}} < 5\% \\ L_i = \max(y_i^1) - \min(y_i^1) \end{cases} \quad (76)$$

8.4 模型求解

step1. 枚举所有角度, 先以 5° 为步长遍历找到较大区间, 然后在较大区间内以 1° 为步长寻优。

step2. 对整个海域数据作坐标变换, 对 Δx 小于 1 的坐标作横坐标统一化, 将数据分为 n 列。

step3. 对数据进行预处理, 求出任意两列之间的面积, 记为 $M_{i,j}$, 表示第 j 列到第 i 列的总面积。

step4. $f[i]$ 表示第 1 列的最优覆盖方案。

用四元组 (重叠面积 S , 覆盖总面积 S_{sum} , 未覆盖面积 S_d , 总测线长度 l_{sum})

$$f[i] = \min\{f[j] + \text{calc}(i, j)\} \quad (77)$$

$$\text{pre}[i] = j \quad (78)$$

定义 $\text{calc}(i, j)$ 为计算第 j 列到第 i 列的最优方案的函数：

在数据中存在两条垂直 x 轴的直线，第 i 列存在 $\{y_i^1\}, \{h_i^1\}$ ，第 j 列存在 $\{y_i^2\}, \{h_i^2\}$ 。

合并 $\{y\}$ 得到离散化序列，补全离散序列，对 $\{y_i^1\}$ ，所有的 i, x 均满足 $y_{i-1}^1 < y_x^2 < y_i^2$ ， x 表示 i 与 $i-1$ 列中任一列。可以得到 $\{y_x^1\}$ 和 $\{y_x^2\}$ 的表达式如下：

$$\begin{cases} y_x^1 = \frac{h_{i-1}^1 - h_i^1}{y_i^1 - y_{i-1}^1} (y_x^2 - y_{i-1}^1) + h_{i-1}^1 \\ y_x^2 = \frac{h_{i-2}^2 - h_i^2}{y_i^2 - y_{i-2}^2} (y_x^2 - y_{i-1}^1) + h_{i-1}^2 \end{cases} \quad (79)$$

得到可以一一对应的 $\{y_i^1\}$ 与 $\{y_i^2\}$ 。

由于我们采用直线测线，可以是 $\beta = 0^\circ$ ，故长度为 $2\sqrt{3}D$ 。第 i 列和第 j 列，两列可向外延伸的探测宽度分别为 l_i 和 l_j ，横坐标为 x_i 和 x_j 。

可以求得二者的重叠长度为

$$l = \max\{0, l_i + l_j - (x_i - x_j)\} \quad (80)$$

计算得到重叠部分的面积为

$$S = lh = h \max\{0, l_i + l_j - (x_i - x_j)\} \quad (81)$$

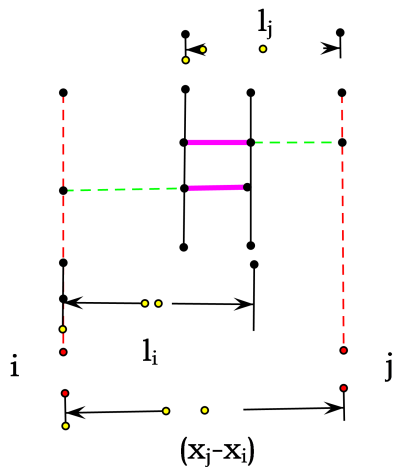


图 25 求解 S 示意图

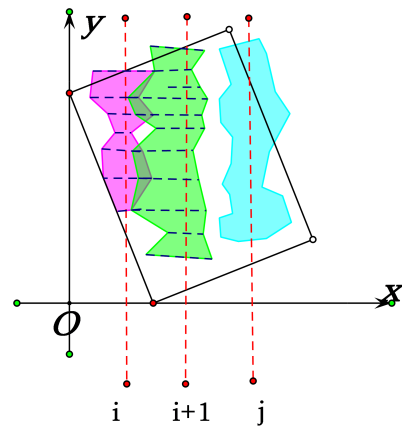


图 26 重叠面积示意图

对一一对应的 $\{y_i^1\}$ 与 $\{y_i^2\}$ 求和可以得到测线探测重叠的面积为：

$$S_q = h \sum_{i=1}^N \sum_{j=i+1}^N (l_i + l_j - (x_i - x_j)) \quad (82)$$

测线探测海底的覆盖总面积为：

$$S_{sum} = h \sum_{i=1}^N l_i + h \sum_{j=i+1}^N l_j - S_q \quad (83)$$

测线未探测到的海底的总面积为：

$$S_d = M(i, j) - S_{sum} \quad (84)$$

测线总长度为：

$$l_{sum} = y_{imax} - y_{imin} \quad (85)$$

step5. 遍历 i ，对所有 $f[i]$ 满足 $\frac{S_{sum}+S_d}{S_{max}} > 0.95$ ，求最优解

step6. 输出方案：

当 $\beta = 124.98^\circ$ 时可以求得整个待测矩形海域的最优一组测线方案。测线总长度为 455134.96 米，漏测海域占总待测海域的 2.69%，在重叠区域中重叠率超过 20% 部分的总长度为 50648.66m。

表 9 $\beta = 124.98^\circ$ 时的最优规划路线

β	测线的总长度/m	漏测海域占	在重叠区域中
		总待测海域面	重叠率超过 20%
		积的百分比	部分的总长度/m
34.98°	455134.96	2.69%	50648.66

九、模型推广与评价

9.1 模型优点

1. 区域测深测线规划模型巧妙利用了解析几何和立体几何的计算，将立体问题转化为平面化的操作。同时采用贪心算法，采用递推的方法得出最后方案，算法效率 $O(n)$ ，算法效率搞。
2. 基于动态规划的多波束测线优化模型采用化规算法，旋转坐标，把所有的计算都转换成直线计算，使计算简单快速。采用动态规划算法，在误差允许的范围内，利于得到全局最优解。

9.2 模型缺点

1. 算法精度存在一定误差：区域测深测线规划模型空间坐标转换使用了大量的三角函数，double 数据存在溢出，存在精度问题。基于动态规划的多波束测线优化模型计算面积时把每个离散点视为单位大小方块，在旋转的方块的边缘位置会超出，得到超过标准面积，在计算覆盖率和总面积的时候存在一定误差。
2. 基于动态规划的多波束测线优化模型算法效率是 $O(n^3)$, n 为每一列上的点数，在算法效率上枚举角度，在算法极限 $n=500$ 每秒可以计算一个角度，算法瓶颈在边上的点数。

参考文献

- [1] Charles E.Leiserson. 算法导论（第三版）.[D/OL]. 机械工业出版社, 2013
- [2] 徐承煌. 余弦定理在三维空间中的推广 [D]. 温州大学, 1991.
- [3] Agostinho Agra; Miguel Constantino. Mixing polyhedra with two non divisible coefficients[J]. Mathematical ProgrammingVolume 135, Issue 1-2. 2012. PP 45-87
- [4] Tang Rui; Luo Biao; Liao Yuxin Supply Chain Optimization Based on Single-Objective Multi-Stage Optimization Model. [J]. Tang Rui; Luo Biao; Liao Yuxin
- [5] 吴焱明, 曹宁. 基于贪心算法的板材多孔加工路径优化算法研究[J]. 合肥工业大学机械工程学院, 2020-12-23

十、附录

10.1 T1

```
#include<bits/stdc++.h>
using namespace std;
const double pi=acos(-1);
double k;
double hudu(double x){
    return x/180*pi;
}
double calc(double X1,double X2){
    return sqrt(k*k+1)*abs(X1-X2);
}
double get_cross(double k1,double b1,double k2,double b2){
    return (b2-b1)/(k1-k2);
}
double X1[10],X2[10],l[10],dep[10],p[10];
int main(){
    int n=9;
    k=tan(hudu(1.5));
    for(int i=1;i<=9;i++){
        double x0=200*(i-5);
        dep[i]=70-k*x0;
        X1[i]=get_cross(tan(pi/6),-tan(pi/6)*x0+70,k,0);
        X2[i]=get_cross(tan(pi/6*5),-tan(pi/6*5)*x0+70,k,0);
        l[i]=calc(X1[i],X2[i]);
        //printf("%.31f %.31f\n",X1[i],X2[i]);
        if(i!=1)
            p[i]=(X2[i-1]-X1[i])/(X2[i]-X1[i]);
    }
    for(int i=1;i<=n;i++)printf("%.21f ",dep[i]);
    puts("");
    for(int i=1;i<=n;i++)printf("%.21f ",l[i]);
    puts("");
    for(int i=2;i<=n;i++)printf("%.21f ",p[i]*100);
    puts("");
    return 0;
}
```

10.2 T2

```
#include<bits/stdc++.h>
using namespace std;
const double pi=acos(-1);
```

```

#define sqr(x) ((x)*(x))
double hudu(double x){
    return x/180*pi;
}
double kt;
double calc(double X1,double X2){
    return sqrt(kt*kt+1)*abs(X1-X2);
}
double get_cross(double k1,double b1,double k2,double b2){
    return (b2-b1)/(k1-k2);
}
double af,bt,gm,thta,a[10];
int main(){
    int n=8;
    af=hudu(1.5);
    for(int i=1;i<=n;i++)
        a[i]=0.3*1852*(i-1);
    for(int jd=0;jd<=315;jd+=45){
        printf("%d:\n",jd);
        double tag;
        if(jd<90||jd>270)tag=1;else tag=-1;
        bt=hudu(jd%180);
        if(bt==0){
            for(int i=1;i<=n;i++)
                printf("%.2lf ",sqrt(3)*(120+tag*tan(af)*a[i]));
            puts("");
            continue;
        }
        if(bt==pi/2){
            kt=tan(af);
            tag=0;
        }else{
            gm=acos(1/(cos(bt)*sqrt(1+sqr(tan(bt)/cos(af)))));
            double k=tan(gm);
            kt=k/tan(bt);
        }
        //printf("%lf\n",tag);
        for(int i=1;i<=n;i++){
            double X1=get_cross(tan(pi/6),tag*tan(pi/6)*a[i]+120,kt,0);
            double X2=get_cross(tan(pi/6*5),tag*tan(pi/6*5)*a[i]+120,kt,0);
            printf("%.2lf ",calc(X1,X2));
            //printf("%.3lf %.3lf\n",X1[i],X2[i]);
        }
        puts("");
    }
    return 0;
}

```

10.3 T3

```
#include<bits/stdc++.h>
using namespace std;
const double pi=acos(-1);
const double L=1852*2,H=1852*4;
#define sqr(x) ((x)*(x))
double u1,u2,l1,l2,bt,h,theta,af,gm,sgm,kt;
double hudu(double x){
    return x/180*pi;
}
double calck1(){
    return tan(sgm)-u1*(cos(sgm)+sin(sgm)*tan(sgm))/(1+u1*sin(sgm));
}
double calck2(){
    return tan(sgm)+u2*(cos(sgm)+sin(sgm)*tan(sgm))/(1+u2*sin(sgm));
}
double calcb(double b1){
    b1+=l1*(cos(sgm)+sin(sgm)*tan(sgm)/(1+u1*sin(sgm)));
    return b1;
}
double calcb1(double b){
    return -l1*(cos(sgm)+sin(sgm)*tan(sgm)/(1+u1*sin(sgm)))+b;
}
double calcb2(double b){
    return l2*(cos(sgm)+sin(sgm)*tan(sgm)/(1+u2*sin(sgm)))+b;
}
bool check (double k1,double b1,double k2,double b2){
    double x=(b2-b1)/(k1-k2);
    double y=k1*x+b1;
    return y<0||x<0;
}
bool pd(double b,double x0,double y0){
    l1=b/tan(sgm)*sin(bt)-(h-tan(pi/6)*b/tan(sgm)*sin(bt))/(theta-tan(pi/6));
    l1*=kt;

    l2=(h-tan(-pi/6)*b/tan(sgm)*sin(bt))/(theta-tan(5*pi/6))-b/tan(sgm)*sin(bt);
    l2*=kt;

    //printf("%.2lf: %lf %lf %lf %lf\n",b,u1,l1,u2,l2);
    double k1=calck1();
    double b1=calcb1(b);

    return k1*x0+b1<y0;
```

```

}
double find(double x0,double y0){
    double l=-1e9,r=H;
    while(abs(l-r)>0.001){
        double mid=(l+r)/2;
        if(pd(mid,x0,y0))
            l=mid;
        else r=mid;
    }
    return l;
}
}
struct node{
    double x,y;
}out[10000],pt[10000];
int main(){
    freopen("data.out","w",stdout);
    af=hu du(1.5);
    printf("%lf\n",tan(af));
    h=110+tan(af)*H/2;
    double ans=1e9;
    int aef=-1;
    int cnt=0;
    double kf=-1;
    for(int jd=0;jd<=90;jd++){
        //printf("\n%d:\n",jd);
        int c=0;
        bt=hu du(jd);
        if(jd==90){
            double up=sqrt(3)*(h-tan(af)*H),down=sqrt(3)*h;
            double p=(down-up)/(2*(up+down));
            printf("%d\n",(int)(L/(2*up)));
            printf("%lf\n",p);
            printf("%.2lf\n",H*L/(2*up));
            continue;
        }
        if(bt==0){
            double x1=0,x0,sum=0;
            int cc=0;
            while(x1<H){
                x0=(tan(pi/6)*x1-tan(af)*x1+h)*sqrt(3);
                cc++;
                printf("(0,%.2lf) (%.2lf,%.2lf)\n",x0,L,x0);
                x1=(tan(-pi/6)*x0-h)/(tan(-pi/6)-tan(af));
                sum+=L;
            }
            printf("\n%d %.2lf\n",cc ,sum);
            continue;
        }
    }
}

```

```

}

gm=acos(1/(cos(bt)*sqrt(1+sqr(tan(bt)/cos(af)))));
theta=tan(gm)/tan(bt);
kt=sqrt(sqr(theta)+1);
sgm=acos(cos(gm)*cos(bt));

u1=tan(pi/6)*tan(bt)/cos(bt)/(theta-tan(pi/6))+tan(bt)/cos(bt);
u1*=kt;
u2=-tan(-pi/6)*tan(bt)/cos(bt)/(theta-tan(-pi/6))-tan(bt)/cos(bt);
u2*=kt;

double k=tan(sgm);
double b2,b1,b;
double k1=calck1();
double k2=calck2();
double lb=-1,tb=-1;
//printf("%.21f %.21f %.21f\n",k1,k,k2);
double x0=L,y0=0,res=0;
if(k1<0)y0=H;
bool flag=true;
while(x0>0){
    b=find(x0,y0);

    b1=calcb1(b);
    b2=calcb2(b);

    //printf("%.21f %.21f\n%.21f %.21f\n%.21f %.21f\n",k1,b1,k,b,k2,b2);
    double p=0;
    if(lb!=-1&&tb!=-1){
        double xt=(lb-b1)/(k1-k2);
        double yt=k1*xt+b1;
        double X1=-b1/k1,Xl=-lb/k2;
        if(lb>0&&b1>0)p=(lb-b1)/(b-tb)*xt/L;
        if(lb<0&&b1>0)p=(yt/H*lb+xt/L*X1)/(b-tb);
        if(lb<0&&b1<0)p=yt/H*(X1-Xl)*k/(b-tb);
        //printf("%.21f %.21f\n",lb,b1);
        lb=b2;tb=b;
    }else lb=0,tb=0;
    p=0.1+(p*10-(int)(p*10))/10;
    //printf("%.21f\n",p);
    double r=min(L,(H-b)/k);
    double l=max(0.0,-b/k);
    //printf("%1f %1f\n",l,r);
    res+=(r-l)*sqrt(k*k+1);
    //printf("(%.21f,%.21f)--->(%21f,%.21f)\n",l,k*l+b,r,k*r+b);

```

```

        //printf("%lf %lf\n",x0,y0);
        //printf("%lf %lf %lf\n",b1,b,b2);
        x0=min(L,(H-b2)/k2);
        x0=max(0.0,x0);
        y0=k2*x0+b2;
        //      printf("%lf %lf %lf %lf\n",out[c].ur,out[c].ul,out[c].dr,out[c].dl);
        //      double t=(out[c-1].ur-out[c].ul+out[c-1].dr-out[c].dl);
        //      double p=(out[c].dr-out[c-1].dl)/2;
        //      if(t!=0)p/=t;else t=1e9;
        //      printf("%lf\n",p);
        //      if(c==1)p=0.2;
        out[++c]={x0,y0};
        if(check(k1,b1,k2,b2)||k2>k||c>300){
            flag=false;
            break;
        }
    }
    res=res*cos(gm);
    if(ans>res&&flag){
        ans=res;
        aef=jd;
        cnt=c;
        kf=k;
        for(int j=1;j<=cnt;j++){
            pt[j]=out[j];
        }
        //printf("%lf\n",res);
    }
    printf("%d\n",cnt);
    //printf("%lf\n",kf);
    printf("%lf %d\n",ans,aef);
    for(int i=1;i<=cnt;i++){
        double m=pt[i].y-kf*pt[i].x;
        double xf,yf;
        if(m>0)yf=m,xf=0;
        else xf=-m/kf,yf=0;
        printf("(%.2lf,%.2lf)-->(%f,%f),pt[i].x,pt[i].y);
        //double p=(out[i].dr-out[i-1].dl)/(2*(out[i-1].ur-out[i].ul+out[i-1].dr-out[i].dl));
        //printf("%.2lf ",p*100);
    }
    return 0;
}

```

10.4 T4

```

#include<bits/stdc++.h>
using namespace std;
const int N=5e5+5;
const double dlt=1852*0.02;
const double pi=acos(-1);
double hudu(double x){
    return x/180*pi;
}
struct node{
    double sw,len,sz,sc;
    bool operator < (const node &rhs)const{
        double p1=sw/sc,p2=rhs.sw/rhs.sc;
        if((p1<=0.1)^(p2<=0.1))return p1<p2;
        if(len!=rhs.len)return len<rhs.len;
        return sc*rhs.sz<rhs.sc*sz;
    }
}f[N];
struct T{
    double x,y,v;
    bool operator < (const T &rhs)const{
        return x==rhs.x ? y<rhs.y:x<rhs.x;
    }
}v[N],h[505][505];
int pre[N],hnum[N];
double e[N],al[N],ar[N],hl[N],hr[N],sum[N];
double get_len(int l,int r){
    int cnt=0;
    int tl=0,tr=0;
    double res=0;
    for(int i=1;i<=hnum[l];i++)e[++cnt]=al[++tl]=h[l][i].y;
    for(int i=1;i<=hnum[r];i++)e[++cnt]=ar[++tr]=h[r][i].y;
    sort(e+1,e+cnt+1);
    cnt=unique(e+1,e+cnt+1)-e-1;
    /* puts("e:");for(int i=1;i<=cnt;i++)printf("%lf ",e[i]);puts("");
    puts("al:");for(int i=1;i<=tl;i++)printf("%lf ",al[i]);puts("");
    puts("ar:");for(int i=1;i<=tr;i++)printf("%lf ",ar[i]);puts("");
    */
    int p=1,q=1;
    for(int i=1;i<=cnt;i++){
        while(al[p]<e[i]&&p<tl)p++;
        if(al[tl]<e[i]||al[tl]>e[i])continue;
        if(h[l][p].y==e[i])hl[i]=h[l][p].v;
        else{
            double k=(h[l][p+1].v-h[l][p].v)/(al[p+1]-al[p]);
            hl[i]=k*(e[i]-al[p])+h[l][p].v;
        }
    }
}

```

```

        while(ar[q]<e[i]&&q<tr)q++;
        if(ar[tr]<e[i]||ar[1]>e[i])continue;
        if(h[r][q].y==e[i])hr[i]=h[r][q].v;
        else{
            double k=(h[r][q+1].v-h[r][q].v)/(ar[q+1]-ar[q]);
            hr[i]=k*(e[i]-ar[q])+h[r][q].v;
        }

        double ctp=sqrt(3)*hr[i]*2;
        double tmp=sqrt(3)*(hl[i]+hr[i])-(h[r][q].x-h[l][p].x)*dlt;
        //printf("%lf \n",tmp);
        if(tmp>0.4*ctp)res+=(e[i]-e[i-1])*dlt;
    }
    return res;
}

node calc(int l,int r){
    int cnt=0;
    int tl=0,tr=0;
    node res={sum[r]-sum[l],0,0,0};
    for(int i=1;i<=hnum[l];i++)e[++cnt]=al[++tl]=h[l][i].y;
    for(int i=1;i<=hnum[r];i++)e[++cnt]=ar[++tr]=h[r][i].y;
    sort(e+1,e+cnt+1);
    cnt=unique(e+1,e+cnt+1)-e-1;
    int p=1,q=1;
    for(int i=1;i<=cnt;i++){
        while(al[p]<e[i]&&p<tl)p++;
        if(al[tl]<e[i]||al[1]>e[i])continue;
        if(h[l][p].y==e[i])hl[i]=h[l][p].v;
        else{
            double k=(h[l][p+1].v-h[l][p].v)/(al[p+1]-al[p]);
            hl[i]=k*(e[i]-al[p])+h[l][p].v;
        }
        while(ar[q]<e[i]&&q<tr)q++;
        if(ar[tr]<e[i]||ar[1]>e[i])continue;
        if(h[r][q].y==e[i])hr[i]=h[r][q].v;
        else{
            double k=(h[r][q+1].v-h[r][q].v)/(ar[q+1]-ar[q]);
            hr[i]=k*(e[i]-ar[q])+h[r][q].v;
        }
        double tmp=sqrt(3)*(hl[i]+hr[i])-(h[r][q].x-h[l][p].x)*dlt;
        if(tmp>0)res.sc+=tmp*(e[i]-e[i-1])*dlt;
    }
    for(int i=2;i<=tl;i++)
        res.sz+=sqrt(3)*h[l][i].v*(al[i]-al[i-1])*dlt;
    for(int i=2;i<=tr;i++)
        res.sz+=sqrt(3)*h[r][i].v*(ar[i]-ar[i-1])*dlt;
    res.sz-=res.sc;
}

```

```

    res.len=(ar[tr]-ar[1])*dlt;
    res.sw-=res.sz;
    if(res.sw<0)res.sw=0;
    // printf("%.11f %.11f %.11f %.11f\n",res.sw,res.len,res.sz,res.sc);
    return res;
}

bool cmp(T a,T b){return a.y<b.y;};
int n,m;double a[251][251],id[N];int out[N];
node add(node a,node b){
    a.sw+=b.sw;
    a.len+=b.len;
    a.sz+=b.sz;
    a.sc+=b.sc;
    a.sz=min(a.sz,50000*dlt*dlt);
    return a;
}

int sol[N];
int main(){
    freopen("data.in","r",stdin);
    freopen("data.out","w",stdout);
    double gma;
    scanf("%d%d",&n,&m);
    for(int i=1;i<=n;i++)
    for(int j=1;j<=m;j++)
    scanf("%lf",&a[i][j]);
    node ans={1e9,1e9,1,1e9};
    int ed;
    for(double the=34.98;the<=34.98;the+=0.01){
        int cnt=0;
        double theta=huDu(the);
        for(int i=1;i<=n;i++)
        for(int j=1;j<=m;j++)
        v[++cnt]={i*cos(theta)-j*sin(theta),i*sin(theta)+j*cos(theta),a[i][j]};
        sort(v+1,v+cnt+1);

        int num=0;
        for(int i=1;i<=cnt;i++){
            if(abs(id[num]-v[i].x)>=1)id[++num]=v[i].x;
            h[num][++hnum[num]]=v[i];
        }
        //for(int i=1;i<=num;i++)printf("%lf ",id[i]);puts("");
        //printf("%d\n",num);
        for(int i=1;i<=num;i++){
            sort(h[i]+1,h[i]+hnum[i]+1,cmp);
            sum[i]=sum[i-1]+(h[i][hnum[i]].y-h[i][1].y)*dlt*dlt;
            f[i]={1e9,1e9,1,1e9};
        }
    }
}

```

```

//printf("sum=%lf %lf\n",50000*dlt*dlt,sum[num]);
f[1]={0,0,0,0};
//node t=calc(0,1);
//printf("%lf %lf %lf %lf\n",t.sw,t.len,t.sc,t.sz);
for(int i=2;i<=num;i++)
for(int j=1;j<i;j++){
    node tmp=add(f[j],calc(j,i));
    //node tp=f[j];
    //puts("");
    //printf("pre= %d->%d %lf %lf %lf %lf\n",j,i,tp.sw,tp.len,tp.sc,tp.sz);
    //printf("add= %d->%d %.1lf %.1lf %.1lf
        %.1lf\n",j,i,tmp.sw,tmp.len,tmp.sc,tmp.sz);
    if(tmp<f[i]){
        f[i]=tmp;
        pre[i]=j;
    }
}

for(int j=2;j<=num;j++){
    //printf("%lf %lf %lf %lf\n",f[j].sw,f[j].len,f[j].sc,f[j].sz);
    if(f[j].sz+f[j].sw>=67000000&&f[j]<ans){
        ans=f[j];
        gma=the;
        ed=j;
        for(int i=1;i<=num;i++)
            out[i]=pre[i];
    }
}

for(int i=1;i<=num;i++)hnum[i]=0;
}

printf("%lf\n",gma);
printf("%lf %lf\n",ans.len,ans.sw/(ans.sw+ans.sz));
int pos=ed;
int kt=0;
while(pos!=0){
    sol[++kt]=pos;
    pos=out[pos];
}

int cnt=0;
double theta=hudu(gma);
for(int i=1;i<=n;i++)
for(int j=1;j<=m;j++)
v[++cnt]={i*cos(theta)-j*sin(theta),i*sin(theta)+j*cos(theta),a[i][j]};
sort(v+1,v+cnt+1);

int num=0;
for(int i=1;i<=cnt;i++){

```

```

        if(abs(id[num]-v[i].x)>=1)id[++num]=v[i].x;
        h[num][++hnum[num]]=v[i];
    }
    for(int i=1;i<=num;i++)
        sort(h[i]+1,h[i]+hnum[i]+1,cmp);
    double res=0;
    for(int i=2;i<=kt;i++)
        res+=get_len(sol[i],sol[i-1]);
    printf("%lf\n",res);

    for(int i=kt;i>0;i--)
        printf("%d ",sol[i]);
    return 0;
}

```